MINISTÉRIO DA DEFESA
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA

Processamento de Imagens - Trabalho Nº 2

Disciplina: Processamento Digital de Imagem II

Mateus Lemos de Abrantes - Cap
André Leonardi Favalessa - Aluno

1. Introdução

O crescimento exponencial na geração e no compartilhamento de dados digitais transformou a maneira como interagimos com a informação visual. Nesse cenário, as imagens digitais representam uma parcela significativa do tráfego global de dados, tornando o armazenamento e a transmissão eficientes um desafio computacional crítico. A compressão de imagens surge como uma solução fundamental para este problema, buscando reduzir a quantidade de dados necessários para representar uma imagem digital sem, idealmente, degradar sua qualidade visual de forma perceptível [Gonzalez e Woods 2010].

Este trabalho foca na análise teórica e prática de dois dos mais influentes algoritmos de compressão sem perdas (*lossless*): a Codificação de Huffman e o algoritmo Lempel-Ziv-Welch (LZW). A Codificação de Huffman é um método estatístico que atribui códigos de comprimento variável e prefixo-livre aos dados, baseando-se em suas probabilidades de ocorrência para alcançar a otimização da codificação [Huffman 1952]. O LZW, por sua vez, é um método baseado em dicionário que constrói uma tabela de sequências de dados dinamicamente durante o processo de compressão, sendo particularmente eficaz em imagens com padrões repetitivos [Welch 1984].

Deste modo, tem-se como objetivo implementar e avaliar o desempenho desses algoritmos em diferentes tipos de imagem e solidificar o entendimento sobre os princípios e conceituais que governam a eficiência da compressão de dados.

2. Objetivos

Este trabalho tem como propósito central investigar e avaliar, de forma quantitativa, a eficácia de algoritmos de compressão de imagens sem perdas. A análise comparativa de desempenho é realizada por meio da aplicação prática desses métodos em diferentes representações de imagem.

2.1 Objetivo Geral

Analisar e comparar o desempenho dos algoritmos de compressão de Huffman e Lempel-Ziv-Welch (LZW) em termos de taxa de compressão e tempo de processamento, aplicando-os a imagens fornecidas.

2.2 Objetivos Específicos

1. Pesquisar a fundamentação teórica dos métodos de compressão de dados designados, com base na literatura de referência, focando em suas características, vantagens e desvantagens.

2. Implementar códigos para pré-processar uma imagem colorida (Lena), gerando suas versões em níveis de cinza (pancromática) e binária (preto e branco) a partir de um limiar.

3. Aplicar os algoritmos de compressão (oHuffman e LZW) sobre o conjunto de imagens, incluindo as três versões da imagem Lena e a imagem específica designada ao grupo.

4. Avaliar e mensurar a performance dos algoritmos, coletando dados de taxa de compressão e tempo de execução para cada um dos testes realizados.

5. Estruturar e apresentar os resultados em tabelas comparativas, permitindo uma análise crítica e conclusiva sobre a eficiência de cada método em relação aos diferentes tipos de imagem.

6. Elaborar um relatório técnico que documente todo o processo, desde a fundamentação teórica até a análise dos resultados e conclusões.

3. Materiais e Métodos

Foi utilizado um ambiente de desenvolvimento e execução configurado conforme as especificações a seguir:

3.1 Hardware

O processamento foi realizado em um computador pessoal equipado com um processador Intel Core i5-10300H de 10ª geração, operando a uma frequência base de 2.50GHz. O sistema conta com 16GB de memória RAM.

O armazenamento é composto por duas unidades de estado sólido (SSD) de alta velocidade, garantindo rápida leitura e escrita de dados: uma unidade Kingston de 465,76Gb e uma unidade Samsung de 476,94Gb.

Para o processamento gráfico, o equipamento utiliza um sistema híbrido: dispõe de gráficos integrados Intel UHD Graphics para tarefas de baixa demanda e economia de energia, e uma placa de vídeo dedicada NVIDIA GeForce GTX 1650 para aplicações que exigem maior performance gráfica ou computação paralela.

3.2 Software

O sistema operacional utilizado foi o Microsoft Windows 11 (64 bits). O ambiente de desenvolvimento empregado para implementação e execução dos códigos foi o Visual Studio Code, utilizando a linguagem Python 3.14.3 no ambiente virtual Pixi.

O trabalho foi dividido nas seguintes etapas: pré-processamento das imagens, a aplicação dos algoritmos de compressão e a subsequente análise de desempenho. A abordagem metodológica foi desenhada para garantir a reprodutibilidade e a clareza na comparação dos resultados.

Cada script foi desenvolvido de forma modular e com um propósito específico. O primeiro script realiza o pré-processamento, gerando as versões em níveis de cinza e binária da imagem "Lena". Os quatro scripts subsequentes executam os algoritmos de compressão de Huffman e LZW sobre cada uma das imagens designadas (as três versões da "Lena"e a imagem específica do grupo). Ao final de cada execução, os resultados — tempo de processamento, tamanho original, tamanho comprimido e taxa de compressão — são sistematicamente coletados, organizados em tabelas com a biblioteca Pandas e salvos em arquivos no formato CSV para análise posterior.

3.2.1 Bibliotecas

Para o processamento e a análise dos dados foram utilizadas as seguintes bibliotecas da linguagem Python: **OpenCV (cv2)**, **NumPy**, **Dahuffman**, **Pandas** e **Time**.

O **OpenCV** foi a principal ferramenta para as operações de I/O (leitura e escrita) de imagens.

A biblioteca **NumPy** foi empregada para a representação das imagens como matrizes multidimensionais (*arrays*), permitindo a manipulação eficiente dos pixels e a realização de cálculos vetoriais.

A biblioteca **Dahuffman** forneceu uma implementação direta e eficiente do algoritmo de Huffman, abstraindo a complexidade da construção da árvore e da geração dos códigos.

A biblioteca **Pandas** foi utilizada para a estruturação dos resultados. Os dados de tempo, tamanho e taxa de compressão foram organizados em *DataFrames* para fácil visualização e exportação.

A biblioteca **Time** foi empregada para a medição precisa do tempo de execução de cada algoritmo de compressão, um dos principais indicadores de desempenho avaliados.

Funções da biblioteca OpenCV (cv2)

As principais funções utilizadas da biblioteca OpenCV foram:

1. `cv2.imread(caminho, flag)`: Responsável por carregar a imagem digital para a memória. Foi utilizada para ler tanto as imagens originais (`.png`, `.tif`) quanto as imagens geradas no pré-processamento.

2. `cv2.cvtColor(imagem, cv2.COLOR_BGR2GRAY)`: Foi empregada para converter a imagem Lena do padrão de cores BGR (padrão do OpenCV) para uma representação em níveis de cinza (pancromática).

3. `cv2.threshold(cinza, limiar, maxval, tipo)`: Aplicada sobre a imagem em tons de cinza para gerar a versão binária.

4. `cv2.imwrite(caminho, imagem)`: Responsável por salvar as imagens geradas (as versões em níveis de cinza e binária da Lena) em disco.

Funções da biblioteca NumPy (np)

Nos scripts de compressão, destacam-se as seguintes operações da biblioteca NumPy:

1. `ndarray.ndim`: Atributo utilizado para verificar o número de dimensões da matriz da imagem, permitindo distinguir entre imagens com múltiplos canais (ex: RGB, `ndim=3`) e imagens de um único canal (ex: tons de cinza, `ndim=2`).

2. `ndarray.shape`: Atributo que retorna as dimensões da imagem (altura, largura, canais). Foi fundamental para calcular o tamanho original da imagem em bits.

3. `ndarray.flatten()`: Método crucial para preparar os dados para a compressão. Ele transforma a matriz 2D (de um canal) ou cada uma das matrizes de canal da imagem RGB em um vetor unidimensional, que é o formato de entrada esperado pelos algoritmos de Huffman e LZW.

4. `np.ceil()` e `np.log2()`: Funções matemáticas empregadas na implementação manual do LZW para calcular dinamicamente o número de bits necessários para representar cada código gerado pelo dicionário, otimizando o tamanho final do arquivo comprimido.

Funções das bibliotecas Dahuffman e Pandas

As bibliotecas `dahuffman` e `pandas` foram centrais para a compressão e para a análise dos resultados, respectivamente.

1. `HuffmanCodec.from_data(dados)`: Função principal da biblioteca `dahuffman`. Ela analisa o vetor de pixels de entrada, calcula a frequência de cada símbolo (valor de pixel) e constrói a árvore de Huffman e a tabela de códigos ótimas para aqueles dados.

2. `codec.encode(dados)`: Realiza a compressão efetiva dos dados, convertendo a sequência de pixels original em uma sequência de bytes comprimida, cujo tamanho é então medido.

3. `pd.DataFrame(lista_de_dicionarios)`: Função da biblioteca Pandas utilizada para converter a lista de resultados (que armazena métricas como tempo, taxa de compressão, etc.) em uma tabela estruturada e de fácil leitura.

4. `DataFrame.to_csv(caminho)`: Método empregado para exportar a tabela de resultados gerada pelo Pandas, garantindo o armazenamento e a rastreabilidade dos dados coletados.

3.3 Imagens

Para a realização dos experimentos, foram empregadas duas imagens principais com características distintas.

A primeira imagem, `top_mosaic_09cm_area17.tif`, é um arquivo no formato GeoTIFF, caracterizado como um dado raster de alta resolução. A imagem apresenta dimensões de 2336 pixels de largura por 1281 pixels de altura, com um tamanho em disco de aproximadamente 8,63 MB. Trata-se de uma imagem com profundidade radiométrica de 8 bits sem sinal (Byte), o que restringe os valores digitais de cada pixel ao intervalo de 0 a 255. A composição é formada por três bandas espectrais organizadas no modelo RGB (Vermelho, Verde e Azul). A análise estatística indica que a Banda 1 (Vermelho) apresenta a maior média de intensidade (143,11) e o maior desvio padrão (48,10). As Bandas 2 (Verde) e 3 (Azul) exibem médias de intensidade de 76,95 e 74,72, respectivamente. Observa-se que todas as bandas utilizam quase que integralmente a faixa dinâmica disponível, com valores máximos de 251, 252 e 253.

A imagem *lena-Color.png* é composta por três bandas espectrais no modelo RGB, todas com dados válidos (sem valores NoData). A Banda 1 (Vermelho) apresenta valores digitais variando entre 15 e 255; a Banda 2 (Verde) varia de 3 a 238; e a Banda 3 (Azul) varia de 0 a 255. Observa-se que as Bandas 1 e 3 utilizam integralmente o limite superior da faixa dinâmica disponível (255), enquanto a Banda 2 não atinge o valor máximo possível. Esses resultados confirmam que a imagem explora quase toda a faixa radiométrica de 8 bits (0–255). A referida imagem foi utilizada como base para a geração de duas outras versões através de pré-processamento: uma imagem pancromática (níveis de cinza) e uma imagem binária (preto e branco). A primeira, `lena_cinza.jpg`, é a versão em níveis de cinza (pancromática), com um tamanho de 92 KB. Seus pixels variam entre os valores 10 e 243, com uma intensidade média de 94,82 e um desvio padrão de 44,80, indicando uma boa distribuição de tons ao longo da escala de cinza. A segunda, `lena_binaria.jpg`, é a versão binária, com 72 KB. Esta imagem possui apenas dois valores de pixel: 0 (preto) e 255 (branco). A média de intensidade de 57,73 e o alto desvio padrão de 106,48 refletem a natureza de alto contraste da imagem, que foi gerada a partir de um limiar de 50%. A simplicidade

estatística desta imagem, com apenas dois símbolos, a torna um caso de teste particularmente interessante para avaliar a redundância de codificação.

4. Fundamentação Teórica

Os algoritmos de compressão exploram as redundâncias presentes nos dados. Conforme Gonzalez e Woods, existem três tipos principais de redundância em imagens: a redundância de codificação, a redundância espacial e a redundância psicovisual (ou, como o autor adota, informações irrelevantes) [Gonzalez e Woods 2010].

Os métodos estudados, Huffman e LZW, são classificados como algoritmos de compressão sem perdas, permitindo que a imagem seja reconstruída após a descompressão, sendo matematicamente idêntica à original, bit a bit. Eles atuam primariamente sobre a redundância de codificação e, no caso do LZW, também sobre a redundância espacial.

4.1 Codificação de Huffman

A Codificação de Huffman, desenvolvida por David Huffman em 1952, é um dos mais célebres algoritmos de compressão e um exemplo clássico de codificação estatística [Huffman 1952]. Sua estratégia se baseia em explorar a redundância de codificação, que ocorre quando todos os símbolos de um alfabeto (neste caso, os níveis de cinza de 0 a 255) são representados por códigos de mesmo comprimento, como o código ASCII de 8 bits. A ideia central de Huffman é atribuir códigos de comprimento variável aos símbolos: os símbolos mais frequentes recebem códigos mais curtos, e os símbolos menos frequentes recebem códigos mais longos.

O processo de geração dos códigos de Huffman pode ser descrito em duas etapas principais, conforme detalhado por Gonzalez e Woods [Gonzalez e Woods 2010]:

1. **Construção da Árvore de Huffman:**
   (a) **Passo 1: Análise de Frequência.** O primeiro passo é varrer toda a imagem (ou o conjunto de dados) para contar a frequência de ocorrência de cada símbolo (cada valor de pixel de 0 a 255). A probabilidade de cada símbolo é então calculada.
   (b) **Passo 2: Criação dos Nós Iniciais.** Cada símbolo, com sua respectiva probabilidade, é tratado como um nó folha de uma árvore.
   (c) **Passo 3: Construção Iterativa.** Os dois nós com as menores probabilidades são identificados e combinados para formar um novo nó interno. A probabilidade deste novo nó é a soma das probabilidades de seus filhos. Este processo é repetido iterativamente: a cada passo, os dois nós (ou sub-árvores) de menor probabilidade na lista atual são combinados, até que reste apenas um único nó, o nó raiz, que representa a probabilidade total (igual a 1).
2. **Geração dos Códigos:**
   (a) **Passo 4: Atribuição dos Códigos.** A árvore binária resultante é percorrida a partir da raiz em direção a cada folha. Convencionalmente, atribui-se o dígito '0' para a aresta que leva ao filho da esquerda e '1' para a aresta que leva ao filho da direita (ou vice-versa). O código de Huffman para um símbolo

específico é a sequência de '0's e '1's encontrada no caminho da raiz até a folha correspondente àquele símbolo.

Uma característica essencial dos códigos de Huffman é que eles são **prefixo-livres** (ou *prefix codes*). Isso significa que nenhum código de um símbolo é o prefixo do código de outro símbolo. Essa propriedade garante que a decodificação seja unívoca e instantânea, pois ao ler o fluxo de bits comprimido, o decodificador sabe exatamente onde um código termina e o próximo começa, sem a necessidade de separadores. A Codificação de Huffman é ótima no sentido de que gera o código de menor comprimento médio possível para um dado conjunto de probabilidades.

Algoritmo Lempel-Ziv-Welch (LZW)

O algoritmo Lempel-Ziv-Welch (LZW) é uma técnica de compressão baseada em dicionário, desenvolvida por Terry Welch em 1984, como um aprimoramento dos algoritmos LZ77 e LZ78 de Jacob Ziv e Abraham Lempel [Welch 1984]. Diferente de Huffman, que é um método estático e requer uma análise prévia dos dados, o LZW é um método adaptativo: ele constrói o dicionário de sequências de símbolos dinamicamente, tanto durante a compressão quanto durante a descompressão.

O LZW é particularmente eficaz na exploração da redundância espacial, pois ele identifica e codifica sequências repetidas de pixels, que são comuns em áreas de cor ou textura uniformes em uma imagem. A lógica de funcionamento do LZW é a seguinte:

1. **Inicialização do Dicionário:** O processo começa com um dicionário pré-definido contendo todos os símbolos individuais possíveis. Para uma imagem de 8 bits, o dicionário inicial contém 256 entradas, mapeando cada valor de pixel (0 a 255) para seu próprio código.

2. **Processo de Compressão:** O algoritmo lê a sequência de pixels de entrada e a processa da seguinte forma:

   (a) Uma variável, digamos `W`, é usada para armazenar a "sequência atual", que começa vazia ou com o primeiro pixel.

   (b) O algoritmo lê o próximo pixel, digamos `K`.

   (c) Ele então verifica se a concatenação `W + K` (a sequência atual seguida do novo pixel) já existe no dicionário.

   (d) **Se `W + K` existe no dicionário:** A sequência `W` é atualizada para se tornar `W + K`, e o algoritmo continua para o próximo pixel. Nenhuma saída é gerada neste passo.

   (e) **Se `W + K` não existe no dicionário:** O algoritmo executa duas ações:

      i. Envia para a saída o código correspondente à sequência `W` que já estava no dicionário.

      ii. Adiciona a nova sequência `W + K` ao dicionário, associando-a a um novo código (o próximo código disponível).

      iii. A sequência `W` é reiniciada para conter apenas o pixel `K`.

3. **Finalização:** Quando todos os pixels de entrada forem processados, o código correspondente à última sequência `W` é enviado para a saída.

O resultado é uma sequência de códigos. Como o dicionário cresce, esses códigos podem representar sequências cada vez mais longas de pixels, levando à compressão. O decodificador, por sua vez, reconstrói a imagem original realizando o mesmo processo de construção de dicionário. Ele lê um código da entrada comprimida, traduz esse código para a sequência de pixels correspondente usando seu dicionário atual e, em seguida, usa a informação do próximo código para adicionar a mesma nova entrada ao seu dicionário que o compressor adicionou. Por isso, não é necessário salvar o dicionário junto com os dados comprimidos. O LZW é a base para formatos de arquivo populares como TIFF e GIF.

4. Resultados Obtidos e Análise

Neste capítulo, são apresentados e analisados os resultados quantitativos obtidos a partir da aplicação dos algoritmos de compressão de Huffman e LZW sobre o conjunto de imagens definido.

4.1 Pré-processamento da Imagem Lena

A primeira etapa do experimento consistiu no pré-processamento da imagem `lena-Color.png` para gerar suas versões em níveis de cinza e binária, conforme os procedimentos d.1 e d.2 da especificação do trabalho. A Figura 1 apresenta as duas imagens resultantes que, juntamente com a versão colorida original, formam o primeiro conjunto de testes.

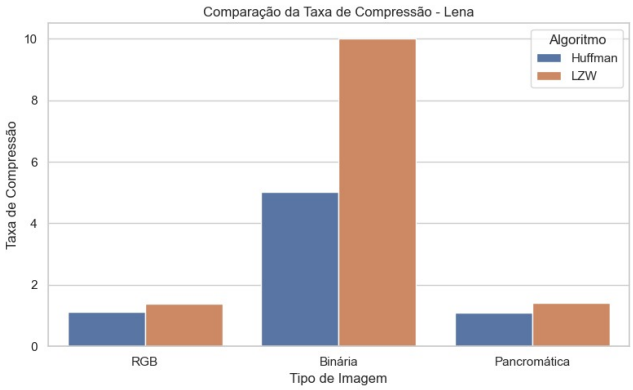
(a) Img. Lena em níveis de cinza. (b) Img. Lena binária (limiar 50%). (c) Img. Lena colorida original.

Figura 1: Imagens geradas a partir do pré-processamento da imagem Lena colorida.

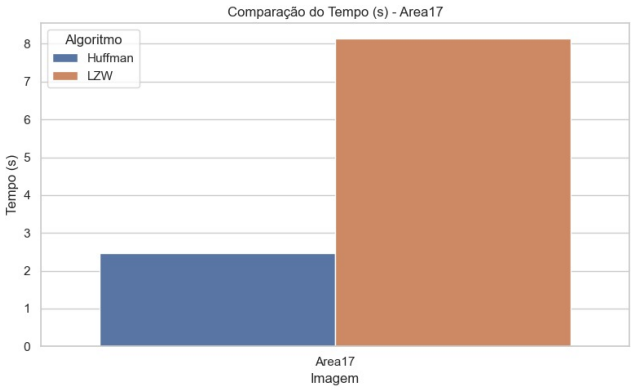
4.1 Análise do Tempo de Execução

Além da taxa de compressão, o tempo de execução é uma métrica crucial para avaliar a eficiência computacional de um algoritmo. As Figuras 2a e 2b apresentam a comparação do tempo, em segundos, gasto por cada algoritmo para processar as imagens do estudo.

A análise dos gráficos revela um padrão consistente e claro em todos os cenários de teste:



(a) Tempo de compressão para as imagens Lena.



(b) Tempo de compressão para a imagem Area17.

Figura 2: Gráficos comparativos do tempo de execução (em segundos).

O algoritmo LZW (barras em laranja) demonstrou ser significativamente mais lento que o Huffman (barras em azul) em todas as imagens testadas. O tempo de execução do LZW chegou a ser de 3 a 4 vezes maior que o do Huffman, tanto para as imagens Lena (Figura 2a) quanto para a imagem de maior dimensão, Area17 (Figura 2b).

Essa diferença de desempenho é uma consequência direta da lógica de funcionamento de cada algoritmo. A Codificação de Huffman, especialmente ao usar uma implementação otimizada, opera em duas fases principais: uma passagem para calcular a frequência dos símbolos e outra para realizar a substituição pelos códigos. É um processo estático e computacionalmente mais simples. Em contrapartida, o LZW realiza um processo dinâmico e mais complexo: para cada pixel (ou sequência de pixels), ele precisa realizar uma busca em um dicionário que está em constante crescimento. Essa operação de busca e inserção a cada passo torna o algoritmo inerentemente mais custoso em termos de processamento, justificando o maior tempo de execução.

A Figura 2b mostra que, para a imagem Area17, que possui dimensões maiores (2336x1281 pixels) em comparação com a Lena (512x512 pixels), o tempo de execução de ambos os algoritmos aumenta consideravelmente, como esperado. O LZW, em particular, levou mais de 8 segundos para processar a imagem, ressaltando que seu custo computacional escala com o tamanho dos dados de entrada.

Em resumo, os resultados demonstram um claro *trade-off* entre os dois métodos: o LZW oferece taxas de compressão superiores ao custo de um maior tempo de processamento, enquanto o Huffman se apresenta como uma alternativa mais rápida, embora geralmente menos eficiente em termos de compressão. A escolha entre um e outro dependeria, em uma aplicação prática, dos requisitos específicos do sistema, como a necessidade de compressão em tempo real versus a otimização máxima do armazenamento.

5. Conclusão

Bibliografia

[Gonzalez e Woods 2010]GONZALEZ, R. C.; WOODS, R. E. *Processamento Digital de Imagens*. 3ª. ed. São Paulo: Pearson Prentice Hall, 2010. ISBN 978-85-7605-153-9.

[Huffman 1952]HUFFMAN, D. A. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, v. 40, n. 9, p. 1098–1101, 1952.

[Welch 1984]WELCH, T. A. A technique for high-performance data compression. *Computer*, v. 17, n. 6, p. 8–19, 1984.